Lecture L04

*Blind review — please score this lecture on row **L04** in the Excel form (1–10 per dimension).*

The Cost of Redundancy: A Motivating Paradox

Imagine a database tracking a university's courses. You store the instructor's phone number every time a student enrolls in their class. If 300 students take "Intro to CS," you store that phone number 300 times.

Now, consider two scenarios: 1. **The Update Nightmare**: The instructor changes their phone number. You must update 300 rows. If the system crashes after 150 updates, your database is now lying to you—half the students have the old number, half have the new. 2. **The Deletion Paradox**: The last student drops the course. You delete the student's enrollment record, and suddenly, the instructor's phone number vanishes from your system entirely.

This is the "Anomaly Problem." Normalization is not about making tables "neat"; it is a mathematical process to ensure that **every piece of data is stored in exactly one place**, eliminating the possibility of these contradictions.

Learning Objectives

By the end of this lecture, you will be able to: 1. **Identify** functional dependencies (FDs) and candidate keys within a given relation schema. 2. **Detect** violations of 1NF, 2NF, 3NF, and BCNF in a database schema. 3. **Decompose** a non-normalized relation into a set of normalized relations to eliminate redundancy and anomalies. 4. **Evaluate** the trade-off between normalization (anomaly prevention) and denormalization (query performance) for specific use cases.

---

1. Foundations: Functional Dependencies and Keys

Before applying normalization, we must establish how attributes relate to one another.

Core Definitions

- **Candidate Key**: A minimal set of attributes that uniquely identifies a tuple.
- **Prime Attribute**: An attribute that is a member of at least one candidate key.
- **Non-prime Attribute**: An attribute that is not part of any candidate key.

Functional Dependencies (FDs)

A functional dependency $X \to Y$ exists if, for every pair of tuples, whenever the values of X are identical, the values of Y **must** also be identical.

The Analogy: The "Lookup Table" Mental Model Think of an FD as a strict rule: "If I know X, there is only one possible value for Y." It is like a dictionary; if you look up the word "Apple" (the determinant), you should get exactly one definition (the dependent). If "Apple" could lead to two different definitions in the same context, the functional dependency is broken.

Cross-Domain Connection: The "Function" in Mathematics This is exactly like a mathematical function $f(x) = y$. For every input x, there is exactly one output y. In database terms, the determinant

X is the input, and the dependent Y is the output. If one input maps to two different outputs, it is no longer a function—and therefore, no longer a functional dependency.

Industry Application: E-Commerce Product Catalogs In a global retail system, consider the relationship between a Global Trade Item Number (GTIN) and a Product Name. Because the GTIN is a worldwide standard, GTIN $\rightarrow$ ProductName is a strict functional dependency. However, ProductName $\rightarrow$ GTIN does not hold, as different manufacturers might name different products "Wireless Mouse."

Example: Identifying FDs Consider EMPLOYEE(EmpID, Name, DeptID, DeptName). 1. EmpID $\rightarrow$ Name: Holds. One ID $\rightarrow$ one Name. 2. DeptID $\rightarrow$ DeptName: Holds. One DeptID $\rightarrow$ one DeptName. 3. Name $\rightarrow$ EmpID: Does **not** hold. Two people named "John Smith" would have different IDs.

---

2. First Normal Form (1NF): Atomicity

The Problem: The "Hidden List"

Consider STUDENT(SID, Name, Courses) where Courses contains {"CS101", "MATH202"}. To find students in "CS101," the database cannot use a simple equality check; it must perform a "string search" or "list scan" inside the cell. This destroys performance and breaks the relational algebra that allows SQL to work efficiently.

Formal Definition

A relation is in **1NF** if every attribute in every tuple contains only **atomic values**.

The Analogy: The Spreadsheet Rule 1NF is the "One Value Per Cell" rule. If you find yourself using commas or semicolons to cram multiple pieces of information into a single cell, you have violated 1NF.

Worked Example: Achieving 1NF

Violation: STUDENT(SID, Name, Courses) where Courses is a set. **Solution**: Decompose into: 1. STUDENT(SID, Name) 2. STUDENT_COURSES(SID, Course)

Real-World Case: Tagging Systems Modern blogging platforms often store "tags" for a post. A naive designer might store tags as a comma-separated string: `Tags: "SQL, Database, Tutorial"`. This violates 1NF. To enable efficient querying (e.g., "Find all posts tagged 'SQL' "), the system must decompose this into a `PostTags` table where each tag is its own row. This allows the database to use B-Tree indexing on the tag column, reducing search complexity from $O(N \times M)$ (where M is string length) to $O(\log N)$.

Crucial Nuance: "Atomic" is relative to the application. A `Date` (YYYY-MM-DD) is atomic if you only ever display the date. It becomes composite if your app needs to filter specifically by `Month`.

Try this yourself: Look at a "User Profile" table. If you have a column `Address` containing "123 Main St, New York, NY 10001," is this in 1NF? It depends on whether your application needs to query by "City" or "Zip Code" independently. If it does, your "atom" is too large.

---

3. Second Normal Form (2NF): Partial Dependencies

The Tension: The Composite Key Trap

Once we have atomic cells, we encounter a new problem: what happens when the "identity" of a row requires two or more columns (a composite key), but some data in that row only relates to *one* of those columns? This creates a "half-dependency" that invites redundancy.

Formal Definition

A relation is in **2NF** if: 1. It is in 1NF. 2. Every non-prime attribute is **fully functionally dependent** on every candidate key.

A violation occurs if a non-prime attribute depends on only a **proper subset** of a candidate key (a **partial dependency**).

The Analogy: The "Wrong Address" Problem Imagine a table ORDER_ITEMS(OrderID, ProductID, Product
The key is {OrderID, ProductID}. The ProductPrice depends on the ProductID, but it doesn't care which OrderID it is in. Storing the price here is like writing the manufacturer's address on every single shipping label—it's redundant and logically misplaced.

Worked Example: Removing Partial Dependencies

Consider TEACH(Student, Course, Instructor). * **Candidate Key**: {Student, Course} * **FD**: Course → Instructor

The Violation: Instructor is non-prime but depends only on Course (a subset of the key). **Decomposition**: 1. STUDENT_COURSE(Student, Course) 2. COURSE_INSTRUCTOR(Course, Instructor)

Industry Application: Warehouse Inventory In a warehouse system, a table STOCK(WarehouseID, ItemID, Wa
has a composite key {WarehouseID, ItemID}. However, WarehouseAddress depends only on WarehouseID. If you store the address in this table, you repeat the address for every single item in that warehouse. Moving the warehouse requires updating thousands of rows. The solution is to move WarehouseAddress to a separate Warehouse table. This is a standard practice in ERP (Enterprise Resource Planning) systems like SAP or Oracle Netsuite to maintain a "Single Source of Truth" for master data.

---

4. Third Normal Form (3NF): Transitive Dependencies

The Tension: The "Middleman" Dependency

Even if every attribute depends on the *whole* key, we can still have redundancy if non-prime attributes depend on *each other*. This is a "hidden" dependency where the key tells us A, A tells us B, and therefore the key tells us B. But B doesn't actually need the key to exist.

Formal Definition

A relation is in **3NF** if: 1. It is in 2NF. 2. There are no **transitive dependencies** of non-prime attributes on candidate keys.

In simpler terms: every non-prime attribute must depend on the key, the whole key, and *nothing but the key*.

The Analogy: The "Chain of Command" Imagine a table EMPLOYEE(EmpID, Name, DeptID, DeptManager).
The EmpID (Key) determines the DeptID. The DeptID then determines the DeptManager.

EmpID → DeptID → DeptManager. The DeptManager is essentially "tagging along" with the employee. If all employees in a department leave, you lose the information of who manages that department. The DeptManager is dependent on the DeptID, not the EmpID.

Worked Example: Removing Transitive Dependencies

Violation: EMPLOYEE(EmpID, Name, DeptID, DeptManager) **Decomposition**: 1. EMPLOYEE(EmpID, Name, 2. DEPARTMENT(DeptID, DeptManager)

Try this yourself: You are designing a system for a library. You have a table BOOK(ISBN, Title, AuthorID, AuthorN Identify the transitive dependency. Which attributes should be moved to a separate table to reach 3NF?

---

5. Boyce-Codd Normal Form (BCNF): The Final Polish

The Tension: The Overlapping Key Conflict

3NF solves most problems, but it has one blind spot: what happens when a table has *multiple* overlapping candidate keys? In these rare cases, a determinant might not be a candidate key, yet it still creates redundancy that 3NF permits.

Formal Definition

A relation is in **BCNF** if for every non-trivial functional dependency $X \to Y$, X is a **superkey**.

BCNF is a stricter version of 3NF. While 3NF allows a non-prime attribute to be dependent on a candidate key, BCNF demands that *everything* that determines something else must be a key.

The Analogy: The "Absolute Authority" Rule If 3NF is about removing "middlemen," BCNF is about ensuring that only the "bosses" (keys) are allowed to make rules. If a non-key attribute starts acting like a boss (determining other attributes), BCNF forces that attribute to become a key in its own table.

Example: The Scheduling Conflict Consider STUDENT_TUTOR(Student, Subject, Tutor) * **Constraints**: 1. For each subject, a student has only one tutor: {Student, Subject} → Tutor. 2. Each tutor specializes in only one subject: Tutor → Subject. * **Candidate Keys**: {Student, Subject} and {Student, Tutor}. * **The Violation**: The FD Tutor → Subject violates BCNF because Tutor is not a superkey.

Decomposition: 1. STUDENT_TUTOR(Student, Tutor) 2. TUTOR_SUBJECT(Tutor, Subject)

Summary: The Normalization Ladder

Form	Requirement	Eliminates…
1NF	Atomic values only	Multi-valued attributes (lists in cells)
2NF	1NF + No partial dependencies	Redundancy from composite keys
3NF	2NF + No transitive dependencies	Redundancy from non-key attributes

Form	Requirement	Eliminates…
BCNF	Every determinant is a superkey	Redundancy from overlapping candidate keys

Final Innovation: The Performance Trade-off In the real world, extreme normalization can lead to "Join Pain." To reconstruct a single record, the database might have to join 10 different tables, which is computationally expensive.

The Engineering Balance: * **OLTP (Online Transaction Processing)**: High normalization (BCNF/3NF) to ensure lightning-fast writes and zero data corruption. * **OLAP (Online Analytical Processing/Data Warehousing)**: Strategic **denormalization**. We intentionally introduce redundancy (like 2NF or 1NF) to eliminate joins and speed up massive read-only queries.